计算机组成原理（2026 年）
简介
前言
概览
实验1 汇编语言与监控程序
  硬件平台模拟环境
  监控程序和终端程序
  RISC-V 指令概况
  RISC-V 汇编语言程序的调试
  实验内容
实验2 Vivado 实验之 ALU 和寄存器堆实验
  SystemVerilog 语言
  通过例子学硬件描述语言
    总述
    加法器
    按钮开关
    秒表
    计数器
    无状态仲裁器
    循环优先级仲裁器
    代码规范
    仿真
  Vivado 使用入门

目录
样例代码
代码编译
模拟器执行与调试
启动和调试监控程序

RISC-V 汇编语言程序的调试

程序调试能力也是编程能力的重要组成部分。本章介绍如何使用 GDB 和 QEMU 来调试 RISC-V 程序。 与一般的应用程序不同，我们需要在裸机环境下，没有操作系统支持的情况下进行程序调试，这就需要模拟器 QEMU 的支持。

在 QEMU 的命令行上，使用 `-s` 来启动一个内置的 gdb server 服务器。这样外部的 gdb 客户端可以连接这个端口（1234）来进行程序调试。也可以使用 `-gdb tcp::9000` 来更改这个服务程序的端口。`-S` 的选项是在 QEMU 启动的时候，先不启动客户虚拟机的 CPU，先等待一个 gdb 客户端连接进来，这样可以调试裸机初始化的过程。

样例代码

下面看一个程序如何编译以及在 QEMU 环境中如何运行。程序很简单，就是计算从 1 加到 10 的总和。

```
.section .text
.globl _start
_start:
    addi t0, x0, 0x1
    addi t1, x0, 0x0
    addi t3, x0, 0xa
loop:
    add t1, t1, t0
    add t0, t0, 0x1
    add t3, t3, -1
    bne t3, x0, loop
    jr ra
```

代码编译

用下面的命令进行编译，生成二进制目标代码：

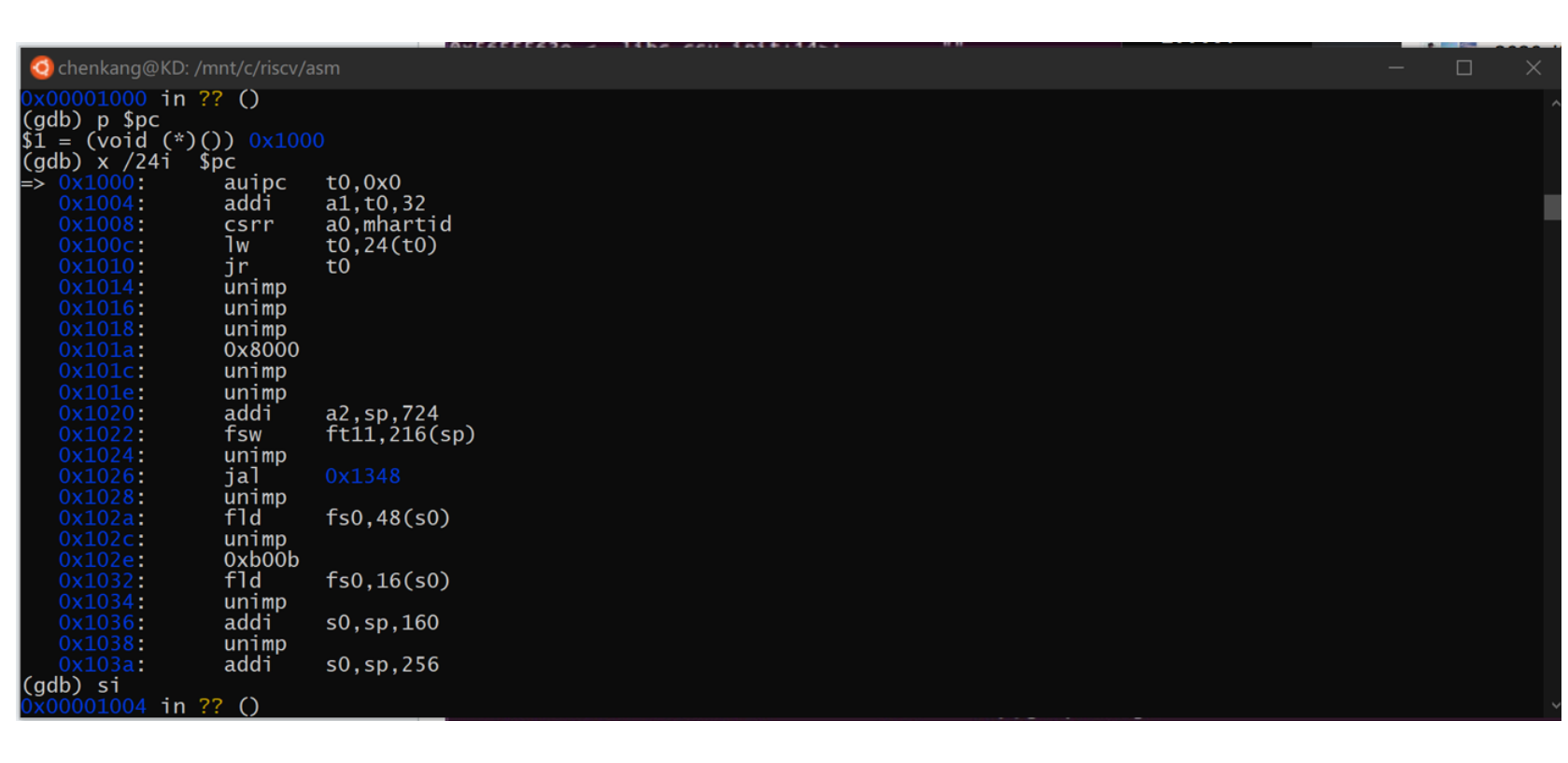
```
riscv64-unknown-elf-c++ -c -nostdlib -nostdinc -static -g -Ttext 0x80000000 -o sum
```

或者生成 32 位格式的二进制目标代码：

```
riscv64-unknown-elf-c++ -nostdlib -nostdinc -static -g -Ttext 0x80000000 sum.s -o
```

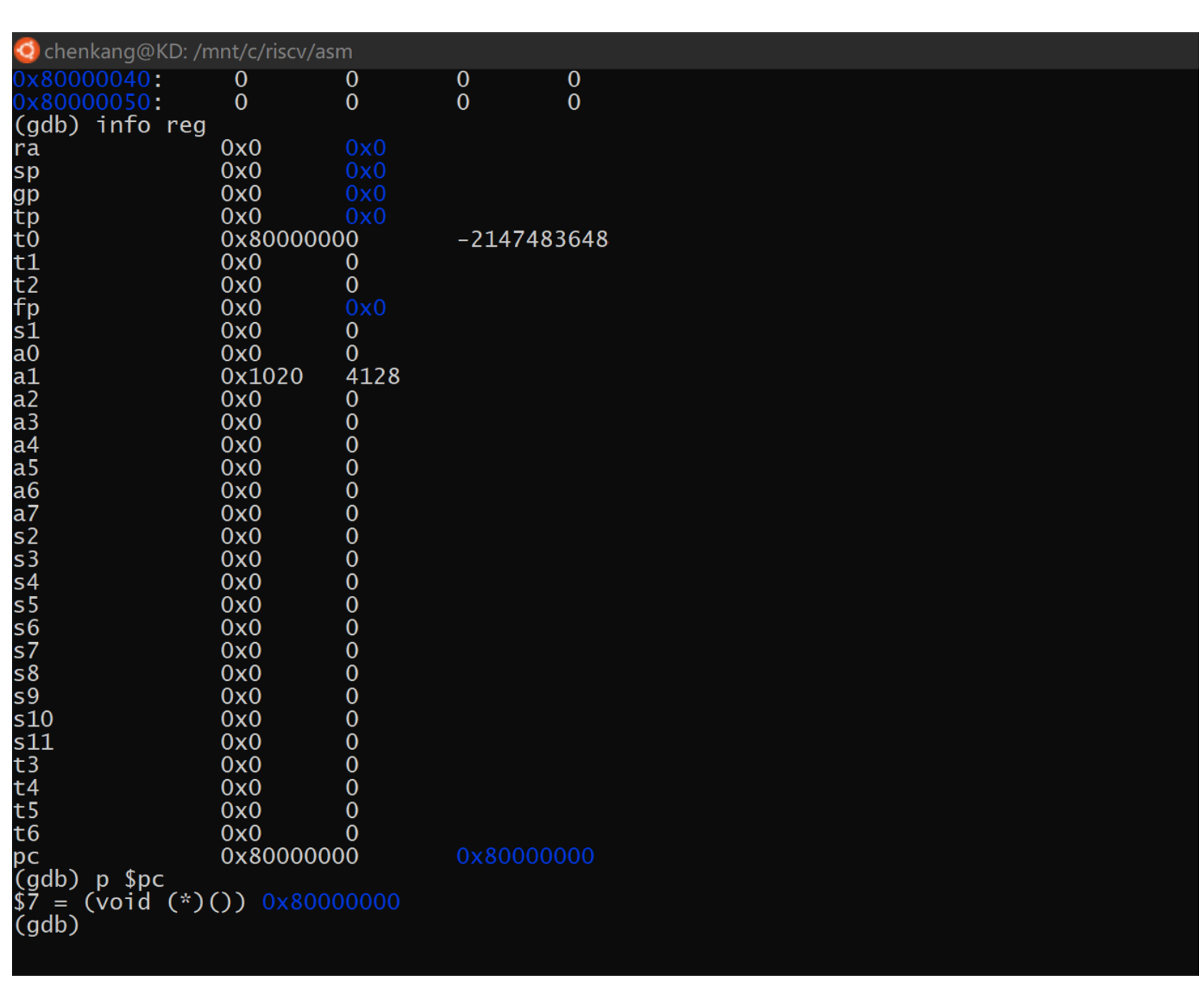
各个编译参数的解释：

- `-nostdlib -nostdinc` 不需要标准库（因为是裸机代码，无标准库的支持）
- `-c` 只进行编译和汇编，不进行链接（因为是裸机代码，也不需要和任何其它库进行链接）
- `-static` 静态编译，不做动态代码的特殊处理
- `-g` 加入调试信息（符号表等），便于调试
- `-o` 输出的文件名
- `-Ttext 0x80000000` 把代码段放到指定地址（模拟器刚刚启动的时候，会从 0x1000 的位置执行，这个位置的代码会把 PC 值指向 0x80000000 位置。这样就需要把用户的代码放到 0x80000000 位置。）（见下面的执行截图）
- `-march=rv32i` 指定使用的 RISC-V 指令集
- `-mabi=ilp32` 指定使用的 abi 结构（指令集是 32 位，不指定这个的话，会使用 64 位的 abi，会不兼容。注意，这里默认使用的是 64 位的编译器。）



```
0x00001000 in ?? ()
(gdb) p $pc
$1 = (void (*)()) 0x1000
(gdb) x /24i  $pc
=> 0x1000:      auipc   t0,0x0
   0x1004:      addi    a1,t0,32
   0x1008:      csrr    a0,mhartid
   0x100c:      lw      t0,24(t0)
   0x1010:      jr      t0
   0x1014:      unimp
   0x1016:      unimp
   0x1018:      unimp
   0x101a:      0x8000
   0x101c:      unimp
   0x101e:      unimp
   0x1020:      addi    a2,sp,724
   0x1022:      fsw     ft11,216(sp)
   0x1024:      unimp
   0x1026:      jal     0x1348
   0x1028:      unimp
   0x102a:      fld     fs0,48(s0)
   0x102c:      unimp
   0x102e:      0xb00b
   0x1032:      fld     fs0,16(s0)
   0x1034:      unimp
   0x1036:      addi    s0,sp,160
   0x1038:      unimp
   0x103a:      addi    s0,sp,256
(gdb) si
0x00001004 in ?? ()
```

执行完上述 5 条指令后的寄存器的内容。



```
0x80000040:     0       0       0       0
0x80000050:     0       0       0       0
(gdb) info reg
ra             0x0      0x0
sp             0x0      0x0
gp             0x0      0x0
tp             0x0      0x0
t0             0x80000000       -2147483648
t1             0x0      0
t2             0x0      0
fp             0x0      0x0
s1             0x0      0
a0             0x0      0
a1             0x1020   4128
a2             0x0      0
a3             0x0      0
a4             0x0      0
a5             0x0      0
a6             0x0      0
a7             0x0      0
s2             0x0      0
s3             0x0      0
s4             0x0      0
s5             0x0      0
s6             0x0      0
s7             0x0      0
s8             0x0      0
s9             0x0      0
s10            0x0      0
s11            0x0      0
t3             0x0      0
t4             0x0      0
t5             0x0      0
t6             0x0      0
pc             0x80000000       0x80000000
(gdb) p $pc
$7 = (void (*)()) 0x80000000
(gdb)
```

t0 就是目标地址，即用户地址，通过 jr t0 跳入。

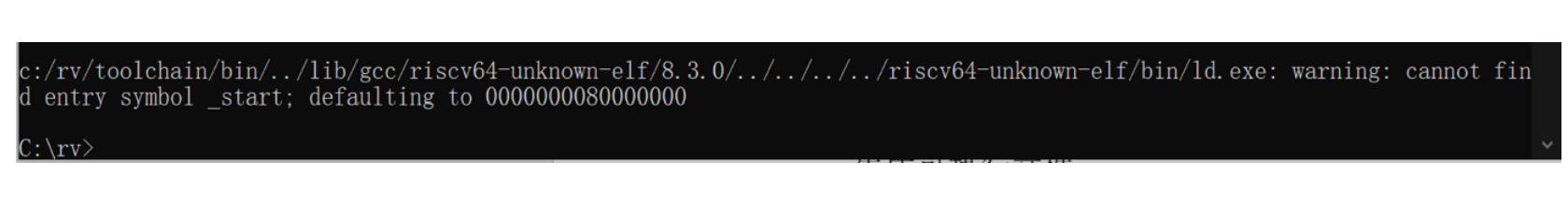
模拟器执行与调试

下面通过编译为 32 位格式的目标代码来看一下程序的执行过程（由于程序不需要任何操作系统的支持，要用远程 gdb 工具进行单条指令执行，观察寄存器值的修改情况。）

上述代码保存为 sum.s，执行如下的命令：

```
riscv64-unknown-elf-c++ -nostdlib -nostdinc -static -g -Ttext 0x80000000 sum.s -o
```

生成可执行文件：

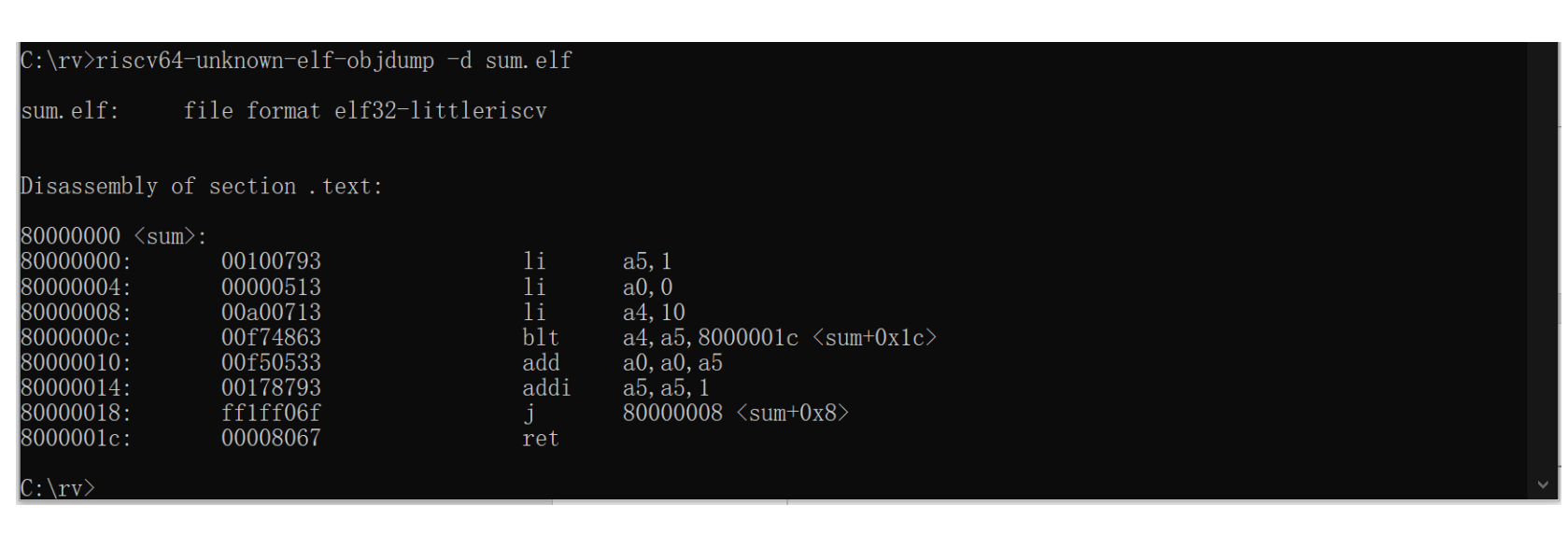


```
c:/rv/toolchain/bin/../lib/gcc/riscv64-unknown-elf/8.3.0/../../../../riscv64-unknown-elf/bin/ld.exe: warning: cannot fin
d entry symbol _start; defaulting to 0000000080000000

C:\rv>
```

生成的可执行文件能在 RISC-V 的 Linux 操作系统下执行，在 Windows 下无法执行。

可以使用 objectdump 来反汇编，看看可执行程序里面的内容：



```
C:\rv>riscv64-unknown-elf-objdump -d sum.elf

sum.elf:     file format elf32-littleriscv


Disassembly of section .text:

80000000 <sum>:
80000000:       00100793                li      a5,1
80000004:       00000513                li      a0,0
80000008:       00a00713                li      a4,10
8000000c:       00f74863                blt     a4,a5,8000001c <sum+0x1c>
80000010:       00f50533                add     a0,a0,a5
80000014:       00178793                addi    a5,a5,1
80000018:       ff1ff06f                j       80000008 <sum+0x8>
8000001c:       00008067                ret

C:\rv>
```

使用 QEMU 模拟器开始装入可执行程序：

```
qemu-system-riscv32 -m 2G -nographic -machine virt -kernel sum.elf -s -S -bios none
```

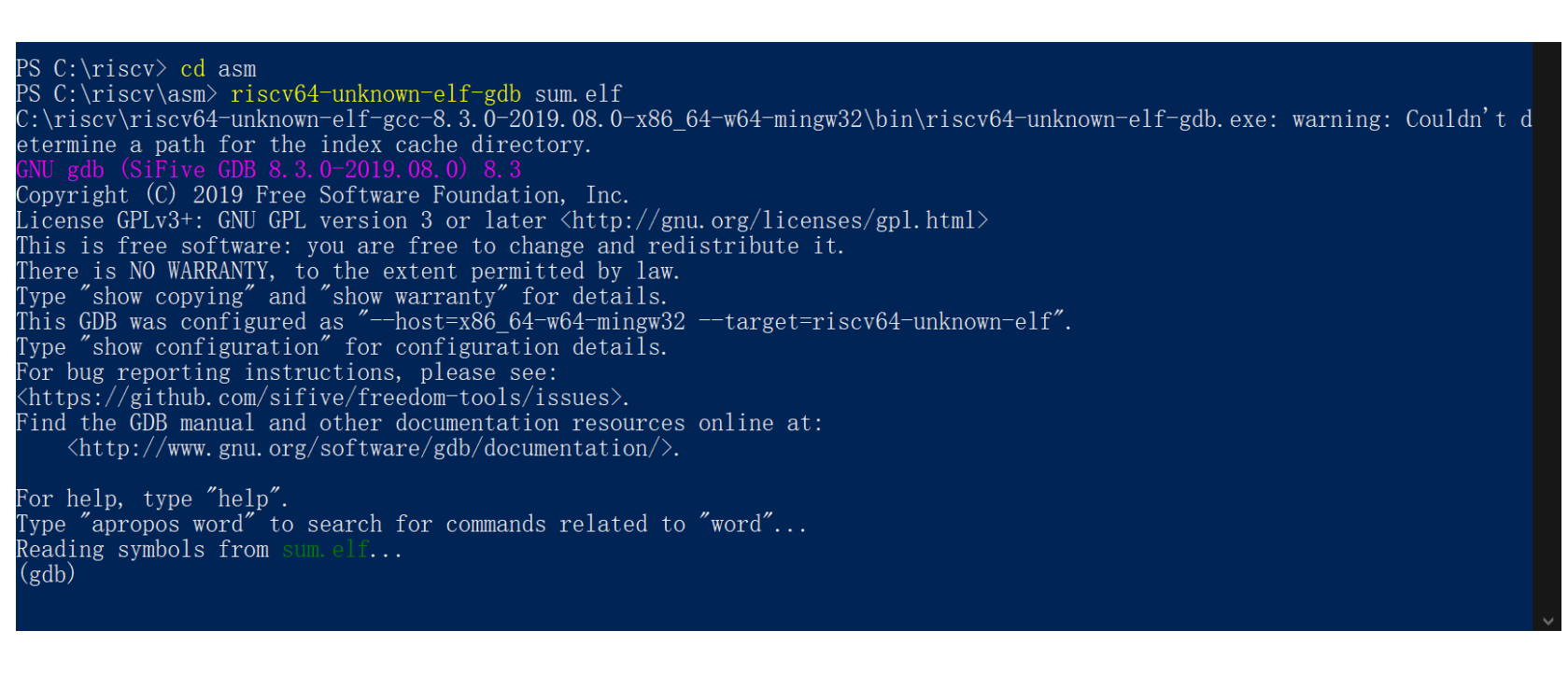
注意一定要带入 `-s -S` 参数。`-s` 的意思是在 QEMU 中启动 gdb server，端口号为 1234，`-S` 的意思是，完成装载之后，不要启动模拟的处理器，等待调试器接入。

另外 `-bios none` 代表的意思就是不需要装载 qemu 默认的基本输入输出系统 bios，在模拟的平台上不需要这部分的信息。基本输入输出系统是机器自带的只读内存 rom 中的二进制代码，用以在机器启动的时候执行，可以用来装入操作系统。

启动 gdb 的调试器客户端：

```
riscv64-unknown-elf-gdb sum.elf
```

后面要跟上 sum.elf，主要目的是两个，一个是 gdb 本身是 64 位的，需要告诉它调试的对象是 32 位的，另外一个是 sum.elf 里面有调试的符号表，在进入调试的时候能够看到被调试的代码。

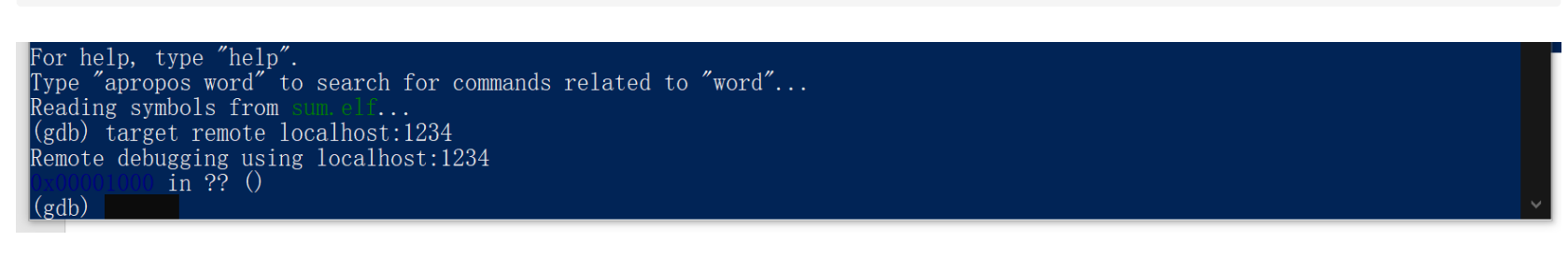


```
PS C:\riscv> cd asm
PS C:\riscv\asm> riscv64-unknown-elf-gdb sum.elf
C:\riscv\riscv64-unknown-elf-gcc-8.3.0-2019.08.0-x86_64-w64-mingw32\bin\riscv64-unknown-elf-gdb.exe: warning: Couldn't d
etermine a path for the index cache directory.
GNU gdb (SiFive GDB 8.3.0-2019.08.0) 8.3
Copyright (C) 2019 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-w64-mingw32 --target=riscv64-unknown-elf".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://github.com/sifive/freedom-tools/issues>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from sum.elf...
(gdb)
```

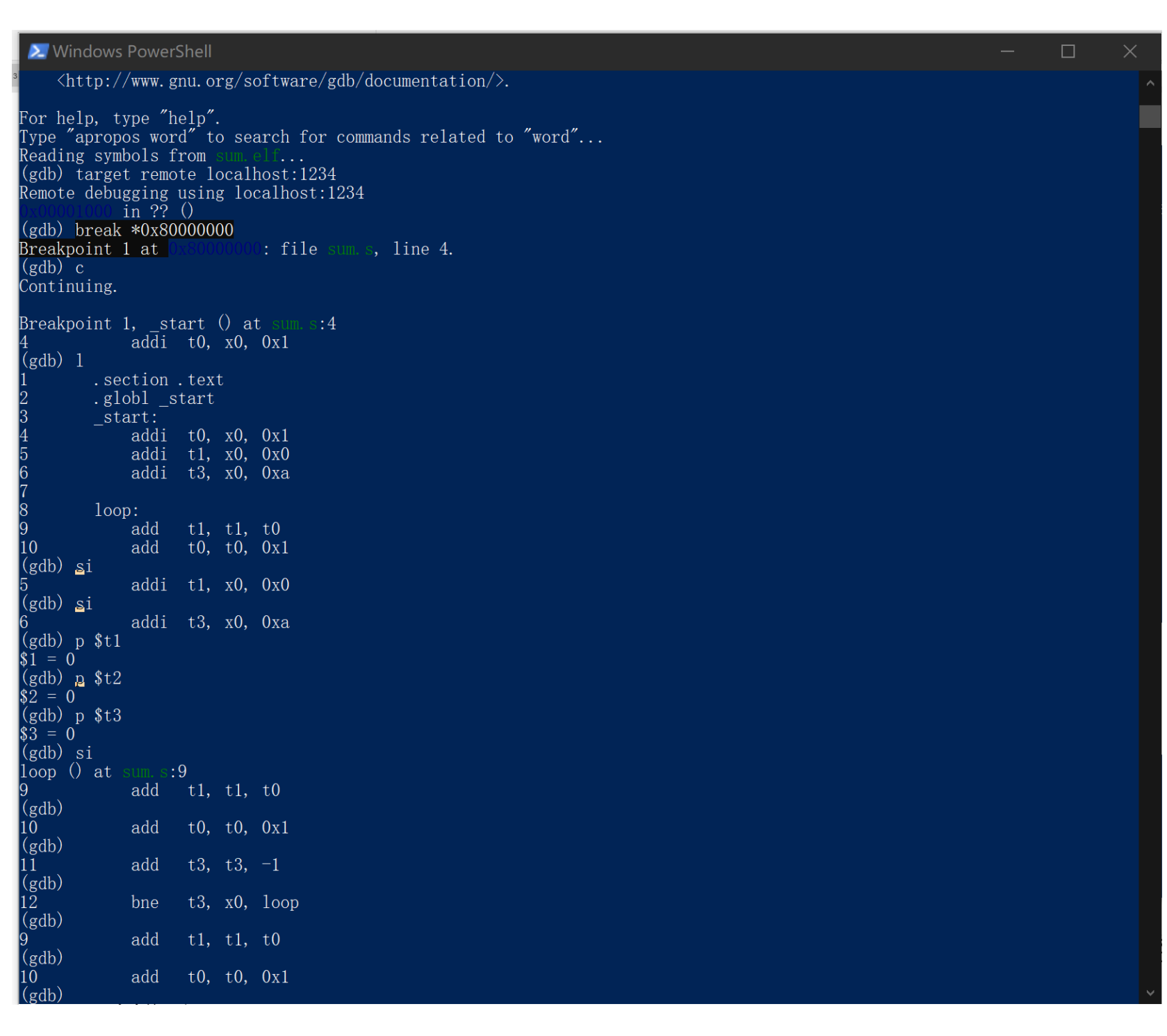
输入调试目标：

```
(gdb) target remote localhost:1234
```



```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from sum.elf...
(gdb) target remote localhost:1234
Remote debugging using localhost:1234
0x00001000 in ?? ()
(gdb)
```

之后可以开始调试，先把断点设置到 0x80000000 的位置，之后可以控制程序单步执行。



```
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from sum.elf...
(gdb) target remote localhost:1234
Remote debugging using localhost:1234
0x00001000 in ?? ()
(gdb) break *0x80000000
Breakpoint 1 at 0x80000000: file sum.s, line 4.
(gdb) c
Continuing.

Breakpoint 1, _start () at sum.s:4
4           addi  t0, x0, 0x1
(gdb) l
1       .section .text
2       .globl _start
3       _start:
4           addi  t0, x0, 0x1
5           addi  t1, x0, 0x0
6           addi  t3, x0, 0xa
7
8       loop:
9           add   t1, t1, t0
10          add   t0, t0, 0x1
(gdb) si
5           addi  t1, x0, 0x0
(gdb) si
6           addi  t3, x0, 0xa
(gdb) p $t1
$1 = 0
(gdb) p $t2
$2 = 0
(gdb) p $t3
$3 = 0
(gdb) si
loop () at sum.s:9
9           add   t1, t1, t0
(gdb)
10          add   t0, t0, 0x1
(gdb)
11          add   t3, t3, -1
(gdb)
12          bne   t3, x0, loop
(gdb)
9           add   t1, t1, t0
(gdb)
10          add   t0, t0, 0x1
(gdb)
```

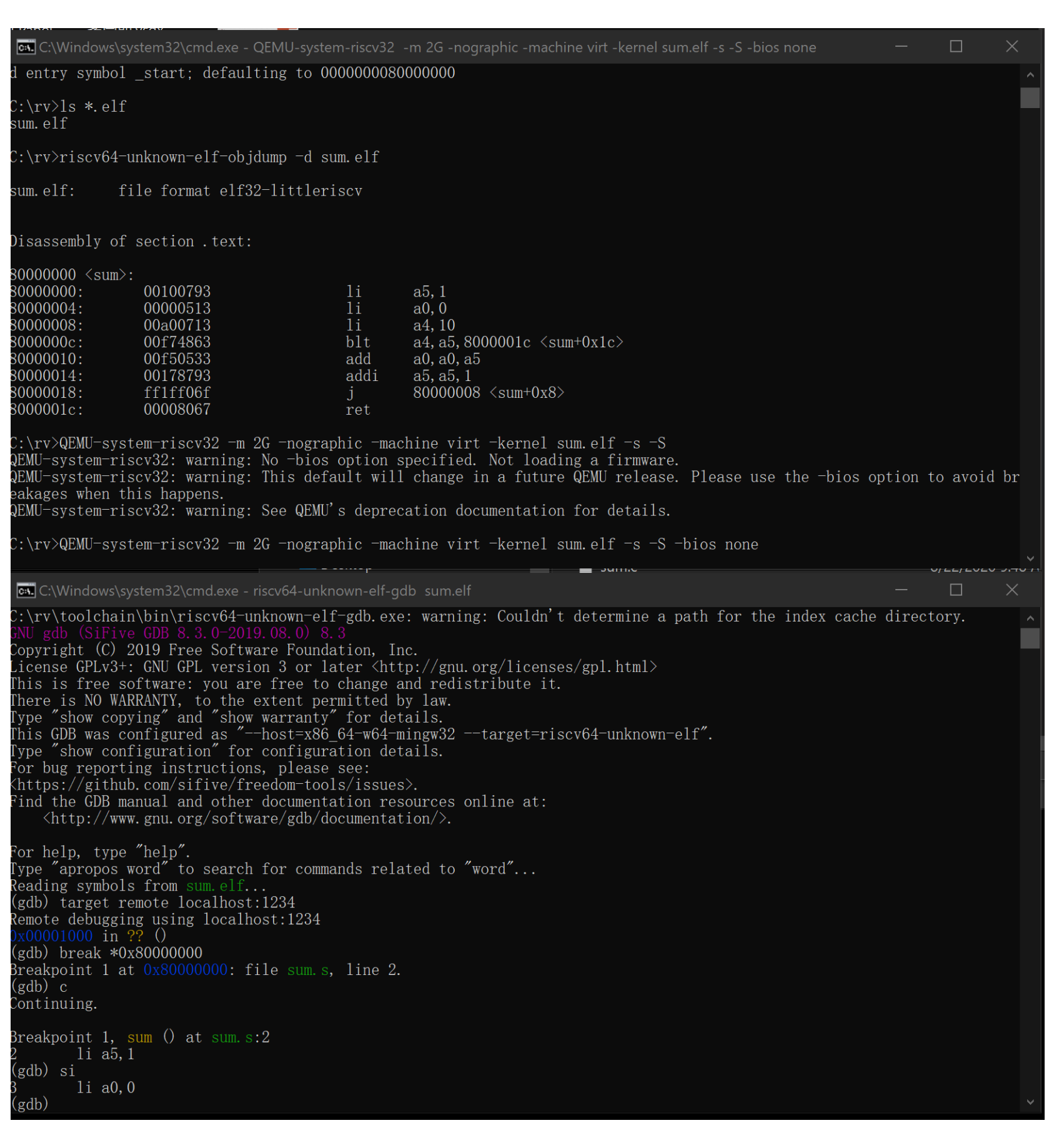
可以通过下面的一些指令来观察程序的执行行为。

gdb 的一些命令：

- `break` 设置断点，比如 `break *0x80000000`
- `disas $pc, $pc+20` 在一个范围内反汇编
- `info reg` 列出所有的寄存器
- `x /24i $pc` 在一个地址之后反汇编 24 条指令
- `x /24d $pc` 在一个地址之后列出内存中 24 字节的内容
- `p $x1` 打印某一个寄存器的内容

容易产生错误的地方：如果 QEMU 和 gdb 互相不能匹配的最大问题就是 32 位和 64 位不匹配。要保证 32 位的 QEMU 模拟器执行 32 位的程序，64 位模拟器去执行 64 位的程序。在调试的时候也要指定正确的字的长度（32 位或者 64 位）。

下面是联合起来的效果：


C:\Windows\system32\cmd.exe - QEMU-system-riscv32 -m 2G -nographic -machine virt -kernel sum.elf -s -S -bios none
C:\Windows\system32\cmd.exe - riscv64-unknown-elf-gdb sum.elf

```
d entry symbol _start; defaulting to 0000000080000000

C:\rv>ls *.elf
sum.elf

C:\rv>riscv64-unknown-elf-objdump -d sum.elf

sum.elf:     file format elf32-littleriscv


Disassembly of section .text:

80000000 <sum>:
80000000:       00100793                li      a5,1
80000004:       00000513                li      a0,0
80000008:       00a00713                li      a4,10
8000000c:       00f74863                blt     a4,a5,8000001c <sum+0x1c>
80000010:       00f50533                add     a0,a0,a5
80000014:       00178793                addi    a5,a5,1
80000018:       ff1ff06f                j       80000008 <sum+0x8>
8000001c:       00008067                ret

C:\rv>QEMU-system-riscv32 -m 2G -nographic -machine virt -kernel sum.elf -s -S
QEMU-system-riscv32: warning: No -bios option specified. Not loading a firmware.
QEMU-system-riscv32: warning: This default will change in a future QEMU release. Please use the -bios option to avoid br
eakages when this happens.
QEMU-system-riscv32: warning: See QEMU's deprecation documentation for details.

C:\rv>QEMU-system-riscv32 -m 2G -nographic -machine virt -kernel sum.elf -s -S -bios none
```

```
C:\rv\toolchain\bin\riscv64-unknown-elf-gdb.exe: warning: Couldn't determine a path for the index cache directory.
GNU gdb (SiFive GDB 8.3.0-2019.08.0) 8.3
Copyright (C) 2019 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-w64-mingw32 --target=riscv64-unknown-elf".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://github.com/sifive/freedom-tools/issues>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from sum.elf...
(gdb) target remote localhost:1234
Remote debugging using localhost:1234
0x00001000 in ?? ()
(gdb) break *0x80000000
Breakpoint 1 at 0x80000000: file sum.s, line 2.
(gdb) c
Continuing.

Breakpoint 1, sum () at sum.s:2
2         li a5,1
(gdb) si
3         li a0,0
(gdb)
```

上面窗口启动 qemu 和用户程序，用命令行启动 qemu 等待连接，注意 `-s` 和 `-S`。下面窗口启动 gdb 命令行进行连接。先设置目标位置 `target remote localhost:1234`。之后设置断点 `break *0x80000000`。随后通过 `c` 命令启动执行，这个时候就会激活上面窗口的 qemu。在执行的时候遇到第一个断点程序会停下来。随后可以通过 `si` 命令进行单步执行。因为没有设置其它的断点，如果使用 `continue` 命令的话会跑飞（即程序执行到无法控制的位置，执行未知的指令）。

另外，程序执行的时候会收到很多环境的影响，这里的一个影响来自于当前目录的 `.gdbinit` 文件。这个文件包含了在启动 `gdb` 时候的一些初始配置，没有注意到的话会造成困扰。可以找一个没有 `.gdbinit` 的目录下面执行上面的命令，避开使用和 kernel 一样的目录。

具体的做法是使用 `打开命令行.cmd` 打开的命令行窗口，然后可以用 `cd` 命令切换到别的目录。这个 cmd 脚本已经设置好一系列所需的环境变量。

启动和调试监控程序

下面过程说明如何使用 gdb 和 QEMU 来调试监控程序：

上述调试 RISC-V 程序的方法也可以应用在调试监控程序上。使用下面命令行可以通过 QEMU 启动监控程序并等待调试：

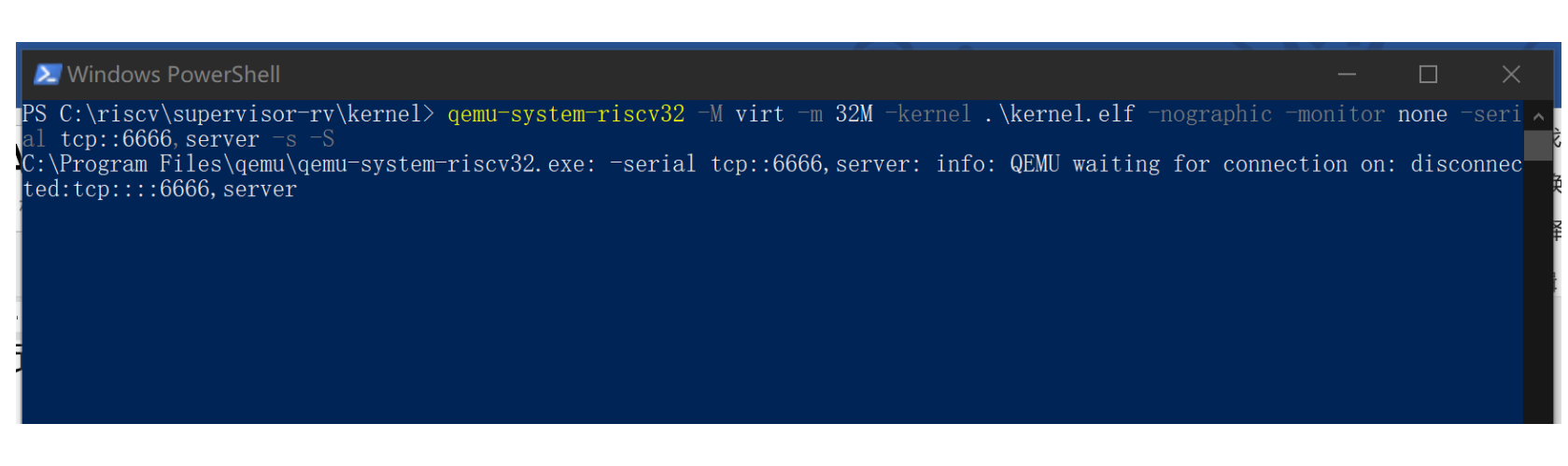
```
qemu-system-riscv32 -M virt -m 32M -kernel ./kernel.elf -nographic -monitor none -s
```

其中几个参数意义是：

- `-M virt` 该参数决定了模拟的 RISC-V 处理器的平台型号，通常不用更改
- `-m 32M` 处理器可访问的内存容量是 32M 字节
- `-kernel ./kernel.elf` 要运行的程序，必须为 elf 格式文件
- `-nographic` 不启用图形功能支持
- `-monitor none` 不启用控制窗口
- `-s -S` 两个参数联合使用，表示启用调试功能，并等待调试器连接（默认的端口是 1234）
- `-bios none` 不加载 BIOS（OpenSBI）

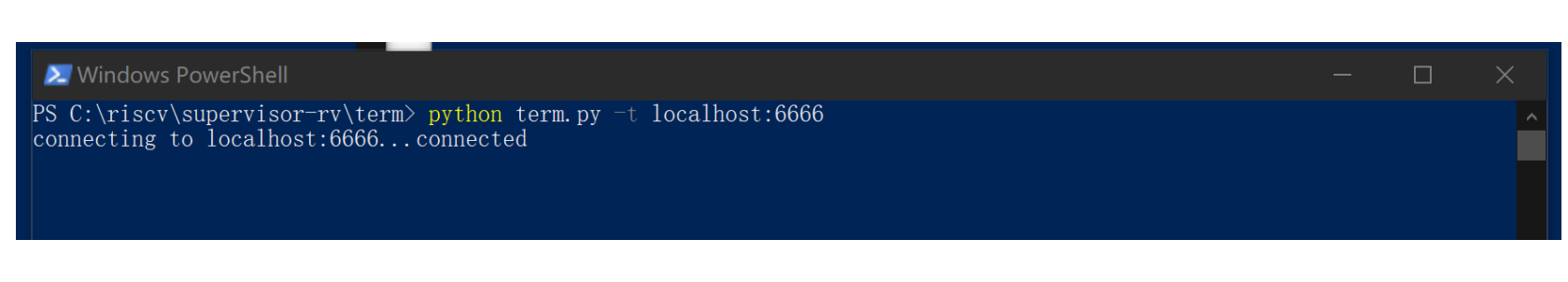
在调试监控程序的时候，QEMU 会打开两个端口，一个端口用于串口通信（端口号：6666），一个端口用作 gdb server 服务程序的端口（端口号：1234）。实际执行时，QEMU 会首先打开串口通信的端口，然后再打开 gdb server 服务程序的端口。所以，如果没有一个终端程序来连到串口的话，QEMU 是不会打开调试端口的。因此，为了能够对监控程序进行调试，需要把终端程序也启动起来。

第一步：使用上述命令来启动 QEMU 模拟器。



```
PS C:\riscv\supervisor-rv\kernel> qemu-system-riscv32 -M virt -m 32M -kernel .\kernel.elf -nographic -monitor none -seri
al tcp::6666,server -s -S
C:\Program Files\qemu\qemu-system-riscv32.exe: -serial tcp::6666,server: info: QEMU waiting for connection on: disconnec
ted:tcp:::6666,server
```

第二步：使用 term 终端程序来连接 6666 号端口。



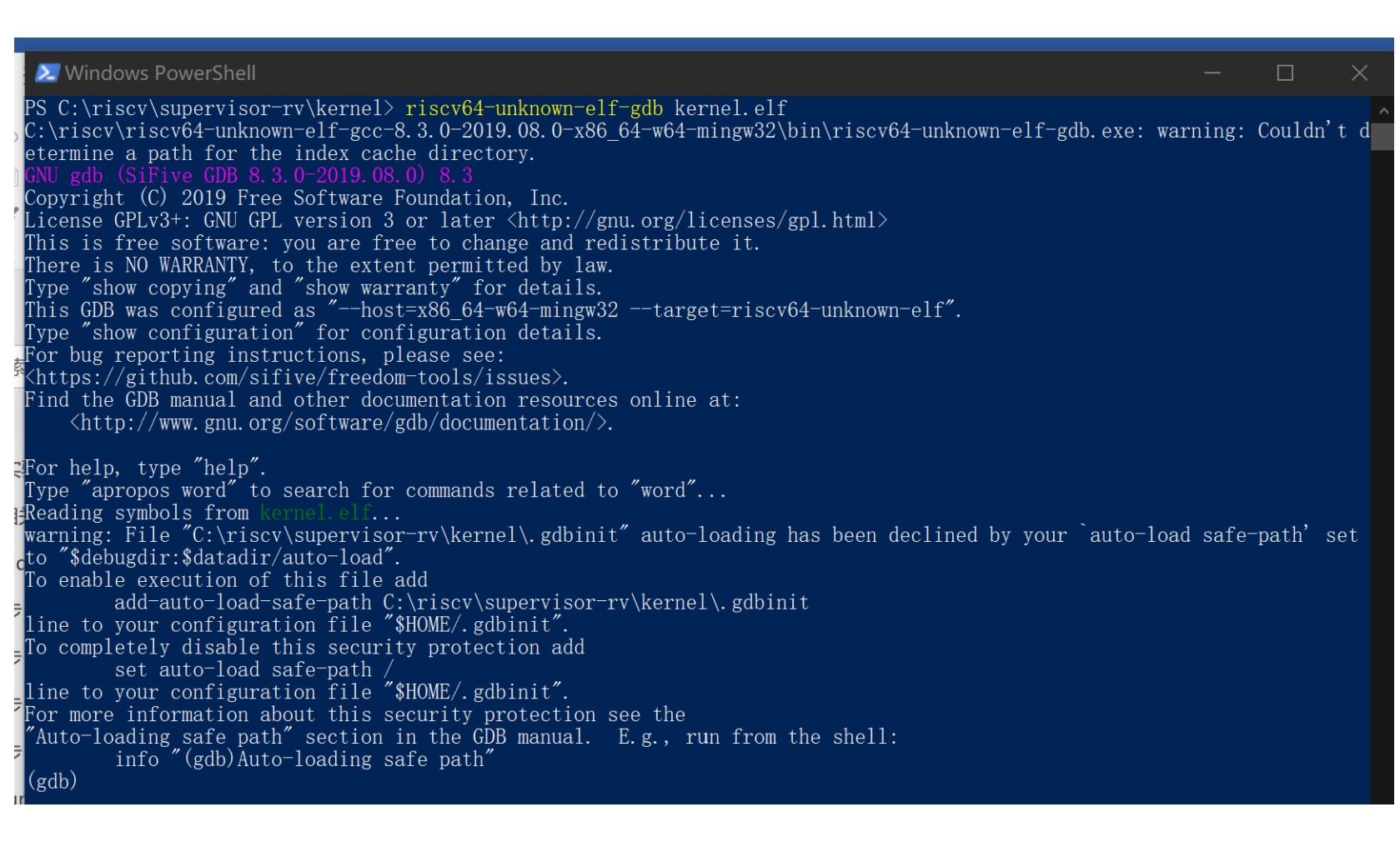
```
PS C:\riscv\supervisor-rv\term> python term.py -t localhost:6666
connecting to localhost:6666...connected
```

可以看到，tcp 已经连接上了。但是，此时监控程序没有起来。因为 QEMU 还在等待 gdb 程序的连接。

第三步：下一步，使用 gdb 连接到 QEMU 的 gdb server。

注意，这里的命令是 `riscv64-unknown-elf-gdb kernel.elf`，后面跟着对应的二进制代码。这样可以通知 gdb 的客户端程序是在服务器端调试那个程序。而且，riscv64-unknown-elf-gdb 默认任务服务器端的程序是 64 位的，但是实际上监控程序是 32 位的。这里通过 `kernel.elf` 的参数明确告诉 gdb 客户端，需要调试的程序是 32 位的，这样双方才能够正确通信。

下图是 gdb 启动之后的情况。

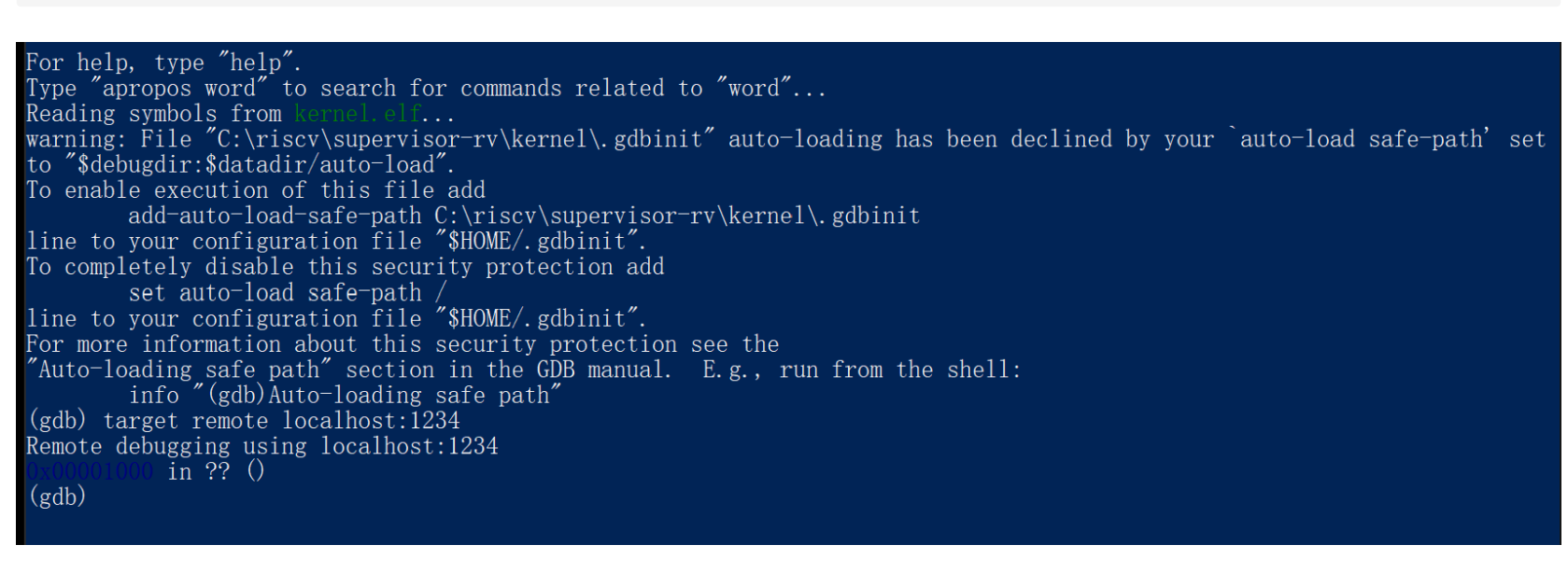


```
PS C:\riscv\supervisor-rv\kernel> riscv64-unknown-elf-gdb kernel.elf
C:\riscv\riscv64-unknown-elf-gcc-8.3.0-2019.08.0-x86_64-w64-mingw32\bin\riscv64-unknown-elf-gdb.exe: warning: Couldn't d
etermine a path for the index cache directory.
GNU gdb (SiFive GDB 8.3.0-2019.08.0) 8.3
Copyright (C) 2019 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-w64-mingw32 --target=riscv64-unknown-elf".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://github.com/sifive/freedom-tools/issues>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from kernel.elf...
warning: File "C:\riscv\supervisor-rv\kernel\.gdbinit" auto-loading has been declined by your `auto-load safe-path' set
to "$debugdir:$datadir/auto-load".
To enable execution of this file add
        add-auto-load-safe-path C:\riscv\supervisor-rv\kernel\.gdbinit
line to your configuration file "$HOME/.gdbinit".
To completely disable this security protection add
        set auto-load safe-path /
line to your configuration file "$HOME/.gdbinit".
For more information about this security protection see the
"Auto-loading safe path" section in the GDB manual.  E.g., run from the shell:
        info "(gdb)Auto-loading safe path"
(gdb)
```

这个时候就可以输入调试的目标了

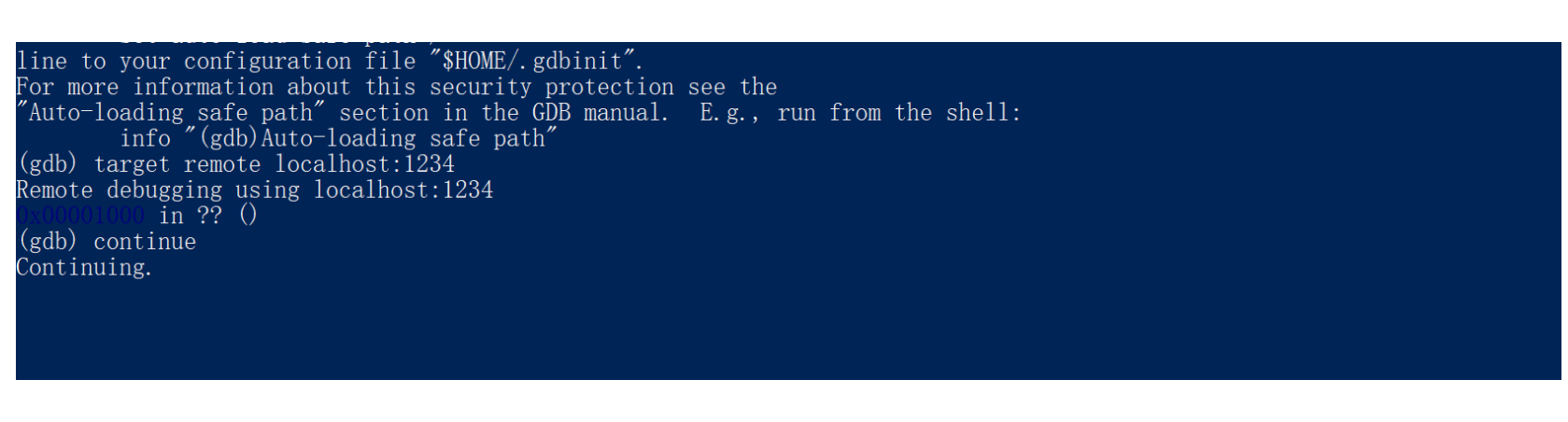
```
(gdb) target remote localhost:1234
```



```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from kernel.elf...
warning: File "C:\riscv\supervisor-rv\kernel\.gdbinit" auto-loading has been declined by your `auto-load safe-path' set
to "$debugdir:$datadir/auto-load".
To enable execution of this file add
        add-auto-load-safe-path C:\riscv\supervisor-rv\kernel\.gdbinit
line to your configuration file "$HOME/.gdbinit".
To completely disable this security protection add
        set auto-load safe-path /
line to your configuration file "$HOME/.gdbinit".
For more information about this security protection see the
"Auto-loading safe path" section in the GDB manual.  E.g., run from the shell:
        info "(gdb)Auto-loading safe path"
(gdb) target remote localhost:1234
Remote debugging using localhost:1234
0x00001000 in ?? ()
(gdb)
```

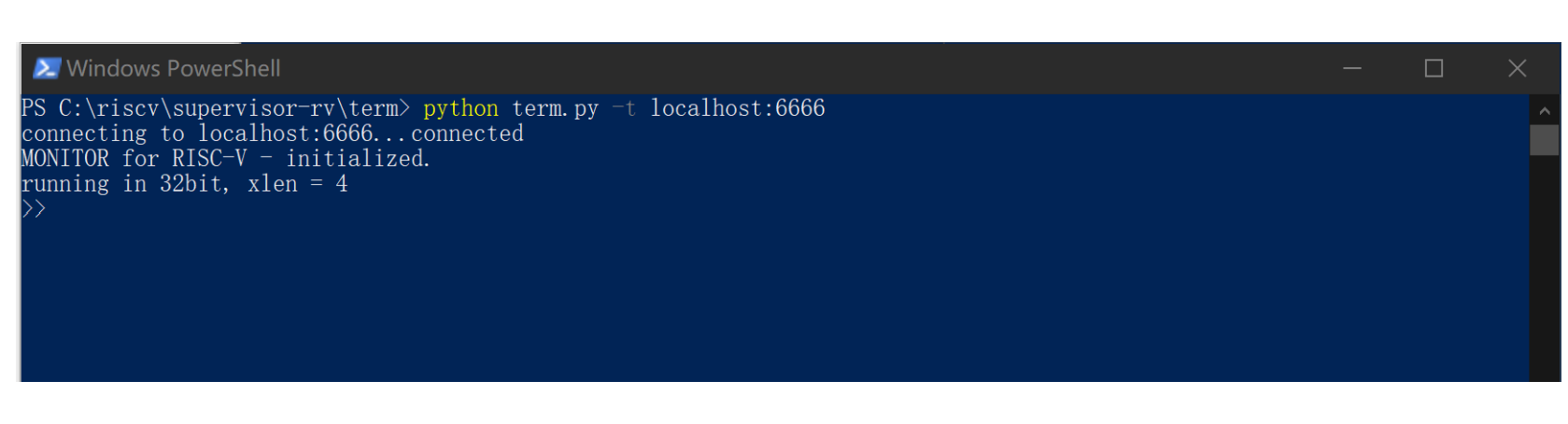
可以看到，gdb 使用 localhost:1234 开始调试。

这时，其它的两个窗口（QEMU 窗口和 term 窗口）没有发生变化，因为这个时候 QEMU 并没有启动模拟的处理器（或者说虚拟机内部的处理器）。在 gdb 窗口中键入 `continue`，启动 QEMU 中的模拟的处理器。



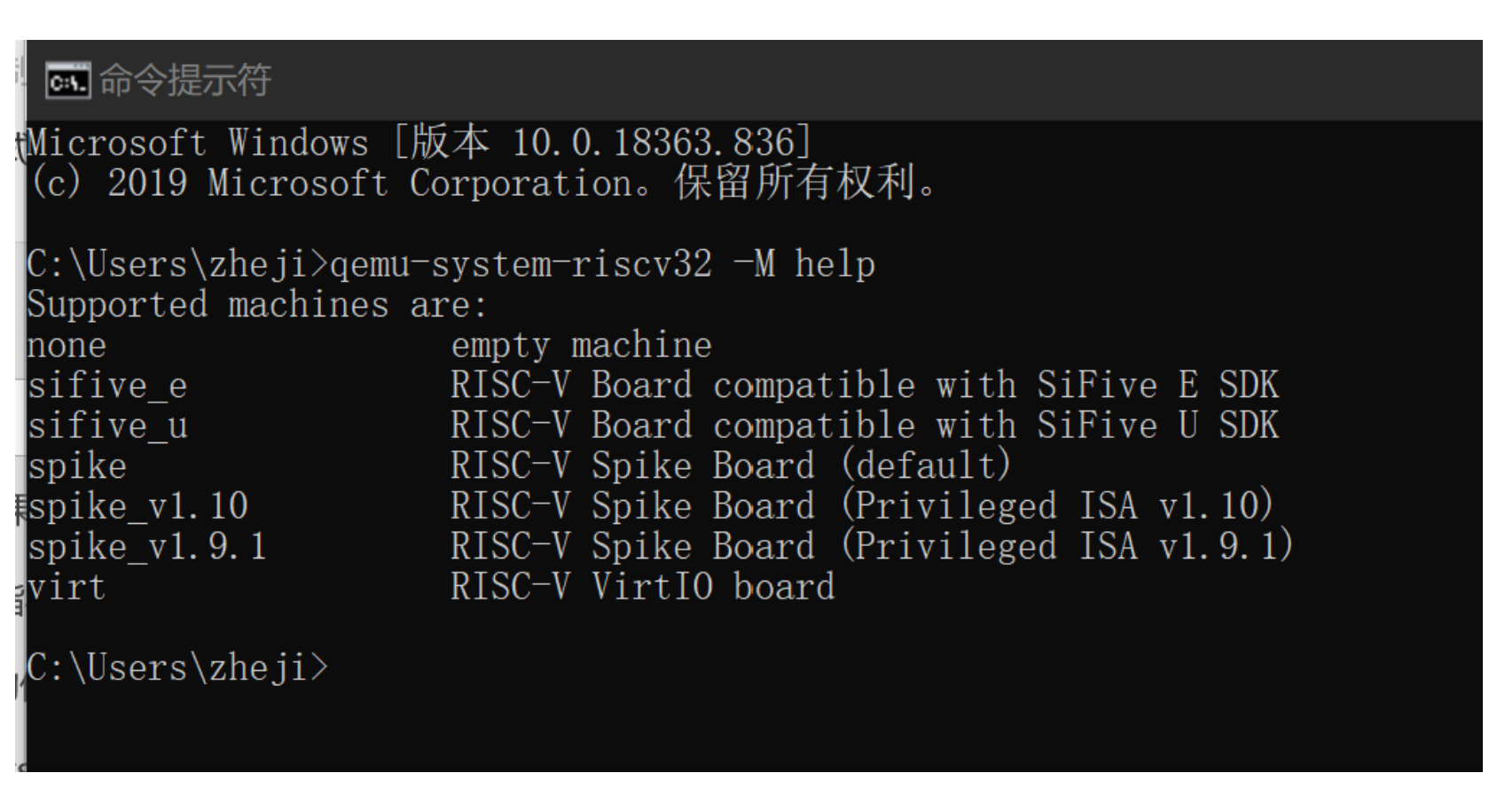
```
line to your configuration file "$HOME/.gdbinit".
For more information about this security protection see the
"Auto-loading safe path" section in the GDB manual.  E.g., run from the shell:
        info "(gdb)Auto-loading safe path"
(gdb) target remote localhost:1234
Remote debugging using localhost:1234
0x00001000 in ?? ()
(gdb) continue
Continuing.
```

终端程序已经可以进行交互了：



```
PS C:\riscv\supervisor-rv\term> python term.py -t localhost:6666
connecting to localhost:6666...connected
MONITOR for RISC-V - initialized.
running in 32bit, xlen = 4
>>
```

关于 `-M` 的解释：



```
Microsoft Windows [版本 10.0.18363.836]
(c) 2019 Microsoft Corporation。保留所有权利。

C:\Users\zheji>qemu-system-riscv32 -M help
Supported machines are:
none                 empty machine
sifive_e             RISC-V Board compatible with SiFive E SDK
sifive_u             RISC-V Board compatible with SiFive U SDK
spike                RISC-V Spike Board (default)
spike_v1.10          RISC-V Spike Board (Privileged ISA v1.10)
spike_v1.9.1         RISC-V Spike Board (Privileged ISA v1.9.1)
virt                 RISC-V VirtIO board

C:\Users\zheji>
```

可以看到，这一参数是指定对应的模拟机器的型号配置，这在实验中不重要，因为我们并不需要其它外设，保持默认的 `virt` 即可。

2024年9月10日　Youyou Lu, gaoyichuan, Kang Chen, Jiajie Chen

作者：Youyou Lu (54.82%), gaoyichuan (26.9%), Kang Chen (14.21%), Jiajie Chen (4.06%)

Copyright © 2020-2026 清华大学计算机组成原理教学组。All Rights Reserved.
Made with Material for MkDocs